

# Python – Testat (FS2026)

## 1 Aufgabenstellung und Bewertungskriterien

- 👉 Implementieren Sie die Klasse `MoveChillData` wie unten im Detail beschrieben und liefern Sie ihr `testat_<vorname_nachname>_fs2026.py`-File mit der darin enthaltenen Klasse **in einem gezippten Ordner mit dem Namen `testat_<vorname_nachname>_fs2026.zip`** der Dozierenden **bis spätestens am Mittwoch, 13.05.2026 um 11:00 Uhr per Moodle** ab.

Dieser Testataufgabe ist ein **gezippter Ordner beigefügt**, welcher alle **für die Aufgabenstellung relevanten Daten** enthält.

**Bewertungskriterien:** Für jede korrekt gelöste Teilaufgabe erhalten Sie Punkte. Die jeweilige Punktzahl ist rechts neben der Aufgabenbeschreibung angegeben. Die maximale Punktzahl für dieses Testat beträgt **26 Punkte**. Direkt unter der Punktzahl finden Sie die zugehörigen Testnummern (T201, T411 usw.). Diese Tests werden mithilfe einer automatisierten Prüferklasse durchgeführt. Nach Abgabe Ihrer Lösung erhalten Sie ein Textfile mit den Ergebnissen aller durchgeführten Tests.

Ein Beispiel für ein solches Auswertungsfile mit Namen

`testat_<vorname_nachname>_auswertungsfiler_fs2026.txt` finden Sie im abgegebenen ZIP-Ordner. Daraus können Sie im Detail entnehmen, welche Aspekte geprüft wurden und wie sich die Teilpunktzahlen zusammensetzen.

**Prüfungszulassung:** Für die Zulassung zur Modulschlussprüfung müssen in diesem Testat **mindestens 40%** der erreichbaren Punkte erzielt werden.

**Hinweis:** Verwenden Sie für die Umsetzung Ihrer Lösung geeignete externe Python-Bibliotheken (z. B. `pandas`, `numpy`, `matplotlib`). Es ist zudem erlaubt, innerhalb Ihrer Klasse eigene Hilfsmethoden zu definieren, um die Logik sinnvoll zu strukturieren und auszulagern. Diese Hilfsmethoden werden nicht direkt geprüft, sondern dienen ausschließlich der internen Organisation Ihrer Implementierung.

**Achtung:** Die Namen der nachfolgend aufgeführten Methoden und Attribute sind zwingend einzuhalten und dürfen nicht verändert werden, da die Testklasse diese explizit verwendet.

## 2 Allgemeines Bewertungskriterium

- Die Klasse und alle zu implementierenden Methoden besitzen aussagekräftige Docstrings.

**1 P.**  
T201

### 3 Einleitung

Die Stadt Zürich testet im Pilotprojekt MOVE AND CHILL intelligente Sitzsensoren, die messen, wie lange Personen auf öffentlichen Sitzgelegenheiten verweilen. Zusätzlich erfassen die Sensoren:

- Temperatur (°C)
- Luftfeuchtigkeit (%)
- Geräuschpegel (dB)
- GPS-Koordinaten
- Timestamp im Format YYYYMMDDhhmmss

Die Daten werden im Abstand von 30 Minuten aufgezeichnet. Im Rahmen dieses Testats soll ein Framework entwickelt werden, welches...

- ... den Datensatz einliest,
- ... den Datensatz bereinigt,
- ... aus dem Datensatz nützliche Features extrahiert, welche für ein späteres ML-Projekt verwendet werden können.

Das Framework soll in Form der Klasse `MoveChillData` implementiert werden.

### 4 Klasse `MoveChillData`

#### 4.1 Methode `__init__(self, csv_path, delimiter = ',')` (8 P)

Folgende Punkte sollen innerhalb der Methode implementiert werden:

- Die Methode nimmt den Pfad `csv_path` sowie das Argument für `delimiter` entgegen, in welchem der Move & Chill Datensatz gespeichert ist, wobei folgendes implementiert werden soll:

- 👉 Das Argument für `csv_path` darf vom Typ `pathlib.Path` oder `str` sein. Falls ein anderer Typ übergeben wird, soll eine `TypeError`-Exception mit der Nachricht `"csv_path must be of type Path or str, got <type>"` ausgelöst werden, wobei für `<type>` der erhaltene Datentyp angegeben werden soll, (via `type(csv_path)`).

3 P.

T411

T412

T413

T414

```
>>> mcd = MoveChillData("taz.view_moveandchill.csv")
>>> mcd = MoveChillData(Path("taz.view_moveandchill.csv"))
>>> mcd = MoveChillData(["taz.view_moveandchill.csv"])
TypeError: csv_path must be of type Path or str, got <class 'list'>
```

- Die Methode liest den M&C Datensatz in ein Pandas DataFrame ein und speichert dieses in der **protected** Instanzvariablen `self._df`, wobei folgende Änderungen vorgenommen werden sollen:

1.5 P.

T415

T416

☞ Der Spaltenname **"zeitpunkt"** soll in **"time"** umbenannt werden.

1 P.

T417

☞ Alle Messzeitpunkte in der Spalte **"time"** sollen in ein **datetime**-Format geparkt werden, wobei dieses wie folgt gewählt werden soll: **"%Y%m%d%H%M%S"**

1 P.

T418

Der Inhalt der ersten 5 Zeilen des geladenen DataFrames sieht nach Instanziierung des Objektes wie folgt aus:

```
>>> mcd._df.head(5)
  objectid  id  sensor_eui  time  temperature  humidity  latitude  longitude  noise  sit  geometry
0      59  133.0  0080E115003BC1DF  2022-09-01 05:12:55      14.9      89.84  47.392551  8.491181  45.0  0  POINT (2679465.5 1249613.2)
1      60  134.0  0080E115003BC1DF  2022-09-01 04:42:55      14.9      89.84  47.392551  8.491181  45.0  0  POINT (2679465.5 1249613.2)
2      61  135.0  0080E115003BC1DF  2022-09-01 04:12:55      14.9      89.84  47.392551  8.491181  45.0  0  POINT (2679465.5 1249613.2)
3      62  136.0  0080E115003BC1DF  2022-09-01 03:42:55      14.9      89.84  47.392551  8.491181  45.0  0  POINT (2679465.5 1249613.2)
4     321   29.0  0080E115003BC1DF  2022-09-01 01:02:42      16.6      84.21  47.392456  8.491221  45.0  0  POINT (2679468.8 1249602.8)
```

Die Datentypen der einzelnen Spalten können mit der `info()`-Methode abgerufen werden:

```
>>> mcd._df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29812 entries, 0 to 29811
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0  objectid        29812 non-null  int64
1  id              29782 non-null  float64
2  sensor_eui      29812 non-null  object
3  time            29812 non-null  datetime64[ns]
4  temperature      29812 non-null  float64
5  humidity         29812 non-null  float64
6  latitude         29812 non-null  float64
7  longitude        29812 non-null  float64
8  noise           29723 non-null  float64
9  sit             29812 non-null  int64
10 geometry       29812 non-null  object
dtypes: datetime64[ns](1), float64(6), int64(2), object(2)
memory usage: 2.5+ MB
None
```

- Die Methode erzeugt des Weiteren **folgende Instanzvariablen**:

1.5 P.

T419

T420

T421

☞ Die **protected** Variable `self._features`, die als **leere Liste initialisiert** werden soll.

☞ Die **protected** Variable `self._feature_analysis`, die als **leeres Dictionary initialisiert** werden soll.

☞ Die **public** Variable `self.report`, die als **leerer String initialisiert** werden soll.

```
>>> mcd._features
[]
>>> print(mcd._feature_analysis)
{}
>>> print(repr(mcd.report))
''
```

## 4.2 Methode `__str__(self)`

(2 P)

Die magische Methode `__str__` soll eine textuelle Zusammenfassung des internen DataFrames (`self._df`) sowie der internen Liste `self._features` erzeugen.

2 P.

T422

T423

Die Darstellung soll folgende Informationen enthalten:

- ☞ die **Shape** des DataFrames mit der Beschreibung **DataFrame shape:**
- ☞ die **Gesamtanzahl** aller NaN-Werte, mit der Beschreibung **Total NaN values:**
- ☞ den aktuellen Inhalt des **Feature-Vektors** (`self._features`), mit der Beschreibung **Extracted features:**

Die einzelnen Angaben sollen jeweils auf **einer neuen Zeile** geschrieben werden:

```
>>> print(mcd)
DataFrame shape: (29812, 11)
Total NaN values: 119
Extracted features: []
```

Inhalt des Strings:

```
>>> print(repr(str(mcd)))
'DataFrame shape: (29812, 11)\nTotal NaN values: 119\nExtracted features: []'
```

## 4.3 Methode `__len__(self)`

(1 P)

Die magische Methode `__len__` soll die Länge des DataFrames `self._df` (= Anzahl Zeilen) zurückgeben:

1 P.

T431

```
>>> len(mcd)
29812
```

## 4.4 Methode `clean(self)`

(3 P)

Diese Methode dient dazu den ins DataFrame geladenen Datensatz zu bereinigen, wozu folgende Punkte gehören:

- ☞ Die Methode **löscht alle Zeilen im DataFrame**, welche NaN enthalten.
- ☞ Die Methode **filtert** das DataFrame so, dass **nur Zeilen erhalten** bleiben, bei denen der Wert in den Spalten `humidity` zwischen 0 % und 100 % und die Werte von `temperature` zwischen -20 °C und 100 °C liegen, die Grenzen **mit einbegriffen**.
- ☞ Die Methode **löscht die Spalten mit Namen** `sensor_eui` sowie `geometry`.

1 P.

T441

1 P.

T442

1 P.

T443

Vorhandene Spalten im DataFrame `self._df` vor Aufruf der Methode:

```
>>> print(mcd._df.head(2))
```

|   | objectid | id    | sensor_eui       | time                | temperature | humidity | latitude  | longitude | noise | sit | geometry                    |
|---|----------|-------|------------------|---------------------|-------------|----------|-----------|-----------|-------|-----|-----------------------------|
| 0 | 59       | 133.0 | 0080E115003BC1DF | 2022-09-01 05:12:55 | 14.9        | 89.84    | 47.392551 | 8.491181  | 45.0  | 0   | POINT (2679465.5 1249613.2) |
| 1 | 60       | 134.0 | 0080E115003BC1DF | 2022-09-01 04:42:55 | 14.9        | 89.84    | 47.392551 | 8.491181  | 45.0  | 0   | POINT (2679465.5 1249613.2) |

humidity-Daten im DataFrame `self._df` vor Aufruf der Methode:

```
>>> print(f"Humidity in DataFrame: min = {mcd._df['humidity'].min()}, max = {mcd._df['humidity'].max()}")
```

Humidity in DataFrame: min = -100.01, max = 103.23

temperature-Daten im DataFrame `self._df` vor Aufruf der Methode:

```
>>> print(f"Temperature in DataFrame: min = {mcd._df['temperature'].min()}, max = {mcd._df['temperature'].max()}")
```

Temperature in DataFrame: min = -100.0, max = 40.91

Ausschnitt von Zeilen mit NaN-Werten im DataFrame `self._df` vor Aufruf der Methode:

```
>>> print(mcd._df[mcd._df.isna().any(axis=1)].head(2))
```

|     | objectid | id      | sensor_eui       | time                | temperature | humidity | latitude  | longitude | noise | sit | geometry                    |
|-----|----------|---------|------------------|---------------------|-------------|----------|-----------|-----------|-------|-----|-----------------------------|
| 116 | 9481     | 16074.0 | 0080E115003BC2AD | 2022-08-18 10:59:11 | 28.37       | 73.92    | 47.392469 | 8.49114   | NaN   | 0   | POINT (2679462.5 1249604)   |
| 414 | 13564    | 11812.0 | 0080E11500180306 | 2022-08-14 03:15:52 | 16.38       | 62.96    | 47.392477 | 8.49036   | NaN   | 0   | POINT (2679403.8 1249604.1) |

Vorhandene Spalten im DataFrame `self._df` nach Aufruf der Methode:

```
>>> mcd.clean()
>>> print(mcd._df.head(2))
```

|   | objectid | id    | time                | temperature | humidity | latitude  | longitude | noise | sit |
|---|----------|-------|---------------------|-------------|----------|-----------|-----------|-------|-----|
| 0 | 59       | 133.0 | 2022-09-01 05:12:55 | 14.9        | 89.84    | 47.392551 | 8.491181  | 45.0  | 0   |
| 1 | 60       | 134.0 | 2022-09-01 04:42:55 | 14.9        | 89.84    | 47.392551 | 8.491181  | 45.0  | 0   |

humidity-Daten im DataFrame `self._df` nach Aufruf der Methode:

```
>>> mcd.clean()
>>> print(f"Humidity in DataFrame: min = {mcd._df['humidity'].min()}, max = {mcd._df['humidity'].max()}")
```

Humidity in DataFrame: min = 16.43, max = 100.0

temperature-Daten im DataFrame `self._df` nach Aufruf der Methode:

```
>>> print(f"Temperature in DataFrame: min = {mcd._df['temperature'].min()}, max = {mcd._df['temperature'].max()}")
```

Temperature in DataFrame: min = 6.42, max = 40.91

Zeilen mit NaN-Werten im DataFrame `self._df` nach Aufruf der Methode:

```
>>> mcd.clean()
>>> print(mcd._df[mcd._df.isna().any(axis=1)])
```

Empty DataFrame  
Columns: [objectid, id, time, temperature, humidity, latitude, longitude, noise, sit]  
Index: []

## 4.5 Methode `create_features(self)`

(3 P)

Die Methode `create_features` soll aus der Zeitspalte des internen DataFrames (`self._df`) zusätzliche Merkmale erzeugen und eine Liste der numerischen Feature-Spalten definieren.

☞ Dazu sollen in einem ersten Schritt aus der Spalte `time` drei neue Spalten erzeugt und ins DataFrame `self._df` eingefügt werden:

- `hour`: enthält die Stunde des Zeitstempels,
- `weekday`: enthält den Wochentag als Zahl (Montag = 0),
- `is_weekend`: enthält den Wert 1, wenn der Wochentag Samstag oder Sonntag ist, sonst 0 (Samstag = 5, Sonntag = 6).

2 P.

T451

T452

Vorhandene Spalten und Daten im DataFrame `self._df` vor Aufruf der Methode:

```
>>> mcd.clean()
>>> print(mcd._df.head(2))
  objectid   id      time  temperature  humidity  latitude  longitude  noise  sit
0         59  133.0  2022-09-01 05:12:55         14.9      89.84  47.392551  8.491181  45.0  0
1         60  134.0  2022-09-01 04:42:55         14.9      89.84  47.392551  8.491181  45.0  0
```

Vorhandene Spalten und Daten im DataFrame `self._df` nach Aufruf der Methode:

```
>>> mcd.clean()
>>> mcd.create_features()
>>> print(mcd._df.head(2))
  objectid   id      time  temperature  humidity  latitude  longitude  noise  sit  hour  weekday  is_weekend
0         59  133.0  2022-09-01 05:12:55         14.9      89.84  47.392551  8.491181  45.0  0     5         3         0
1         60  134.0  2022-09-01 04:42:55         14.9      89.84  47.392551  8.491181  45.0  0     4         3         0
```

☞ Des Weiteren sollen die Spaltennamen des DataFrames `self._df` in die interne Liste `self._features` übernommen werden, wobei **folgende Spaltennamen nicht verwendet** werden sollen: `objectid`, `id`, `time`, `sit`.

1 P.  
T453

Inhalt der Instanzvariablen `self._features` vor Aufruf der Methode:

```
>>> print(mcd._features)
[]
```

Inhalt der Instanzvariablen `self._features` nach Aufruf der Methode:

```
>>> mcd.clean()
>>> mcd.create_features()
>>> print(mcd._features)
['temperature', 'humidity', 'latitude', 'longitude', 'noise', 'hour', 'weekday', 'is_weekend']
```

Die Features sind nun ebenfalls über die magische Methode `__str__` ersichtlich:

```
>>> mcd.clean()
>>> mcd.create_features()
>>> print(mcd)
DataFrame shape: (26562, 12)
Total NaN values: 0
Extracted features: ['temperature', 'humidity', 'latitude', 'longitude', 'noise', 'hour', 'weekday', 'is_weekend']
```

## 4.6 Methode `analyze_features(self)`

(2 P)

Die Methode `analyze_features` soll für alle Feature-Spalten eine einfache Korrelationsanalyse mit der Zielvariable `sit` durchführen.

☞ Dazu wird über alle Einträge in `self._features` iteriert. Für jedes Feature wird der so genannte **Pearson-Korrelationskoeffizient** zwischen der Feature-Spalte im DataFrame `self._df` und der Zielvariable `sit` (ebenfalls in `self._df`) berechnet:

2 P.  
T461

- `pearson_r`: Pearson-Korrelationskoeffizient zwischen `df[col]` und `df["sit"]`. (Siehe Berechnungsvorschrift weiter unten.)

Die Ergebnisse sollen im Dictionary `self._feature_analysis` gespeichert werden, wobei

- **key**: Als **key** im Dictionary der Name des **feature** verwendet werden soll.
- **value**: Als **value** der für das Feature zugehörige Pearson-Korrelationskoeffizient gespeichert werden soll.

Der Inhalt der Instanzvariablen `self._feature_analysis` nach Aufruf der Methode:

```
>>> mcd.clean()
>>> mcd.create_features()
>>> feature_analysis = mcd.analyze_features()
>>> print(mcd._feature_analysis)
{'temperature': np.float64(0.35854067083610863), 'humidity': np.float64(-0.370647801524096),
 'latitude': np.float64(0.021397255835358637), 'longitude': np.float64(0.0445005389435614),
 'noise': np.float64(0.15399995883596465), 'hour': np.float64(0.20913334699790365), 'weekday':
 np.float64(-0.04369579711299858), 'is_weekend': np.float64(-0.024806848558377885)}
```

## Berechnung des Pearson-Korrelationskoeffizienten

Der Pearson-Korrelationskoeffizient misst die **lineare Beziehung** zwischen zwei Variablen  $x$  und  $y$ . Er nimmt Werte im Bereich

$$-1 \leq r \leq 1$$

an, wobei

- Werte nahe 1 = starke positive lineare Korrelation,
- Werte nahe -1 = starke negative lineare Korrelation,
- Werte nahe 0 = keinen linearen Zusammenhang

bedeuten.

Zur Berechnung werden zunächst beide Variablen **zentriert**, indem von **jedem Wert der jeweilige Mittelwert abgezogen** wird:

$$x'_i = x_i - \bar{x}$$

$$y'_i = y_i - \bar{y}$$

Dabei bezeichnet

- $\bar{x}$  den Mittelwert von  $x$
- $\bar{y}$  den Mittelwert von  $y$

Der Pearson-Korrelationskoeffizient ergibt sich dann aus

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

Dabei ist

- $n$  die **Anzahl der Beobachtungen** (= Messpunkte),
- der Zähler die **Kovarianz** der beiden Variablen,
- der Nenner das Produkt ihrer **Standardabweichungen**.

Durch diese Normierung liegt das Ergebnis immer im Bereich  $[-1, 1]$  und ist unabhängig von der Skalierung der Variablen.

## 4.7 Methode `generate_report(self)`

(3 P)

3 P.

T471

T472

T473

Die Methode erzeugt eine textuelle Auswertung der zuvor berechneten Feature-Analyse.

Dazu wird über alle Einträge im internen Dictionary `self._feature_analysis` iteriert, wobei für jedes Feature folgende Informationen ausgelesen werden:

- der Feature-Name (`feature`)
- der berechnete Pearson-Korrelationskoeffizient (`pearson_r`)

☞ Für jedes Feature wird danach eine einfache qualitative Bewertung des Korrelationswertes vorgenommen. Dabei werden folgende Schwellenwerte verwendet:

- **GOOD FEATURE**: wenn  $|r| > 0.3$
- **MODERATE**: wenn  $0.05 \leq |r| \leq 0.3$
- **WEAK - LOW IMPACT**: wenn  $|r| < 0.05$

Danach soll für **jedes Feature eine formatierte Textzeile** erzeugt werden, die den **Namen des Features**, den **Korrelationswert**  $r$  und die **qualitative Bewertung** enthält. Alle Zeilen werden zu einem **mehrzeiligen String** **zusammengefügt**, welcher in der **public Variable** `self.report` **gespeichert** werden soll.

Bei der Formatierung der Zeilen soll folgendes beachtet werden:

- Der Feature-Name soll in einem Feld mit der **Ausgabebreite der Länge des längsten Feature-Namens** fixiert werden.
- Der **Korrelationswert**  $r$  soll auf **drei Nachkommastellen** gerundet werden und das **Vorzeichen** der Zahl soll **immer dargestellt** werden (z. B.  $+0.206$  oder  $-0.374$ ).
- Die einzelnen Teile der Ausgabe werden durch das Zeichen `|` **voneinander getrennt**, wobei links und rechts des Zeichens `|` ein **Leerzeichen** eingefügt werden soll.
- Die Ausgabe soll **aufsteigend nach dem Betrag des Korrelationswertes**  $|r|$  **sortiert** werden.
- Am **Ende jeder Zeile** wird ein **Zeilenumbruch** eingefügt, sodass ein mehrzeiliger Report entsteht.



Der Inhalt der Instanzvariablen `self.report` nach Aufruf der Methode:

```
>>> mcd.clean()
>>> mcd.create_features()
>>> mcd.analyze_features()
>>> mcd.generate_report()
>>> print(mcd.report)
latitude      | r = +0.021 | WEAK - LOW IMPACT
is_weekend    | r = -0.025 | WEAK - LOW IMPACT
weekday       | r = -0.044 | WEAK - LOW IMPACT
longitude     | r = +0.045 | WEAK - LOW IMPACT
noise         | r = +0.154 | MODERATE
hour          | r = +0.209 | MODERATE
temperature   | r = +0.359 | GOOD FEATURE
humidity      | r = -0.371 | GOOD FEATURE
```

Die Instanzvariable `self.report` ist nach Aufruf der Methode nach wie vor ein String:

```
>>> print(type(mcd.report))
<class 'str'>
```

Der Inhalt der Instanzvariablen `self.report` nach Aufruf der Methode:

```
>>> print(repr(mcd.report))
'latitude      | r = +0.021 | WEAK - LOW IMPACT\nis_weekend    | r = -0.025 | WEAK - LOW IMPACT\nweekday       | r = -0.044 | WEAK - LOW IMPACT\nlongitude     | r = +0.045 | WEAK - LOW IMPACT\nnoise         | r = +0.154 | MODERATE\nhour          | r = +0.209 | MODERATE\ntemperature   | r = +0.359 | GOOD\nFEATURE\nhumidity      | r = -0.371 | GOOD FEATURE\n'
```

## 4.8 Methode `plot(self, save_path=None)`

(3 P)

Die Methode erstellt ein **horizontales Balkendiagramm**, das die Korrelationswerte aller Features visualisiert (siehe Abbildung 1).

3 P.

T481

T482

T483

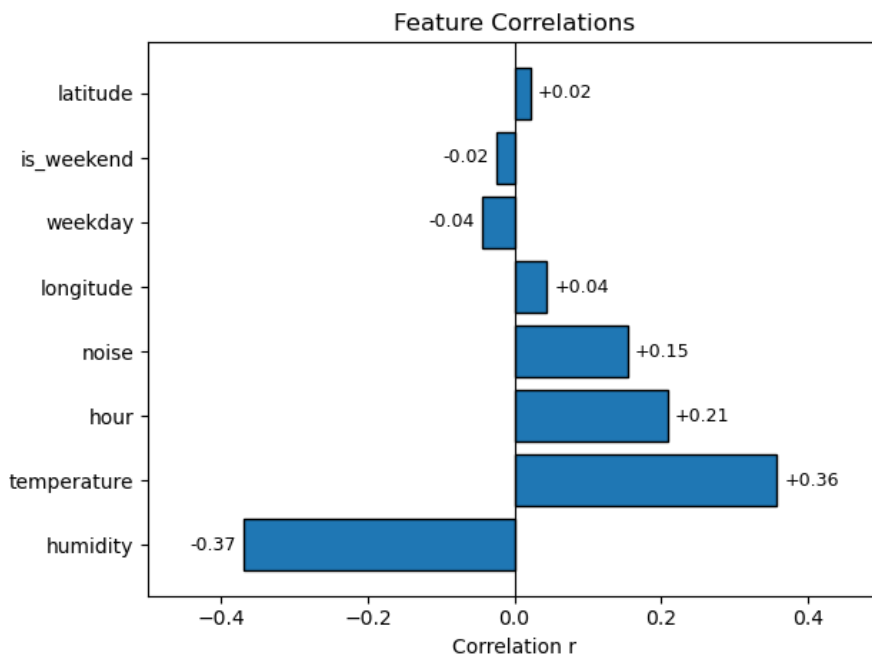


Abbildung 1: Horizontales Balkendiagramm der Feature-Korrelationen.

Vor der Darstellung werden die Features nach dem **absoluten Wert ihrer Korrelation  $|r|$  absteigend** sortiert, sodass die wichtigsten Features oben erscheinen.

☞ Erstellen Sie die Darstellung, wobei folgende Punkte berücksichtigt werden sollen:

- Die Balken werden horizontal gezeichnet (`ax.barh`).
- Der Nullpunkt wird mit einer schwarzen vertikalen Linie visualisiert (`ax.axvline`).
- Bei jedem Balken wird zusätzlich der **Korrelationswert  $r$  als Text auf zwei Kommastellen gerundet** neben dem Balken angezeigt.
- Die Achsen sind wie folgt zu beschriften:
  - **x-Achse:** Correlation  $r$
  - **y-Achse:** Der Name des Features
- Die x-Achse wird auf einen festen Bereich gesetzt (`-0.5 bis 0.5`).
- Der Plot trägt den Titel **Feature Correlations**.
- Optional kann das Diagramm über den Parameter `save_path` als Datei gespeichert werden.
- Die Methode gibt das **Figure**-Objekt zurück.

Nach Ausführung der Methode wird das Diagramm angezeigt und die matplotlib **Figure** zurückgegeben, sodass weitere Anpassungen oder erneutes Speichern möglich sind:

```
>>> fig = mcd.plot(save_path="feature_correlations.png")
>>> print(type(fig))
<class 'matplotlib.figure.Figure'>
```